

USER MANUAL - A SERVERLESS WEB APPLICATION FOR KNOT THEORY: DERIVING THE WIRTINGER PRESENTATION VIA PYSCRIPT

This manual provides a comprehensive guide for users to understand and effectively utilize this serverless web application, available on GitHub¹, to draw knot diagrams and calculate their corresponding Wirtinger presentations and derived topological invariants.

1. KEY FEATURES AND FUNCTIONALITIES

1.1. Interactive Diagram Creation.

- An embedding of the knot can be constructed via a finite succession of straight line segments.
- The user interacts with a digital drawing area, constructing the knot diagram by left-clicking to plot the vertices.
- To close the diagram, the user simply moves the cursor over the first vertex and confirms the action in a modal dialog.

1.2. Intersection Detection and Resolution.

- Once the loop is closed, the algorithm determines all intersections.
- The application systematically highlights each intersection by erasing a small circular area around it.
- To guide the user during this phase, a blinking red dot appears exactly at the crossing currently under consideration.
- The software then prompts the user to specify which segment passes over the other, which is achieved by clicking the “Yes” or “No” buttons.
- If a mistake occurs during this selection process, the user does not need to clear the canvas and redraw the entire diagram. By directly clicking on any previously resolved crossing on the canvas, the application instantly toggles which arc passes over, ensuring a highly fluid and forgiving user experience.

¹<https://tacioli.github.io/Knot-Group-Presentation/>

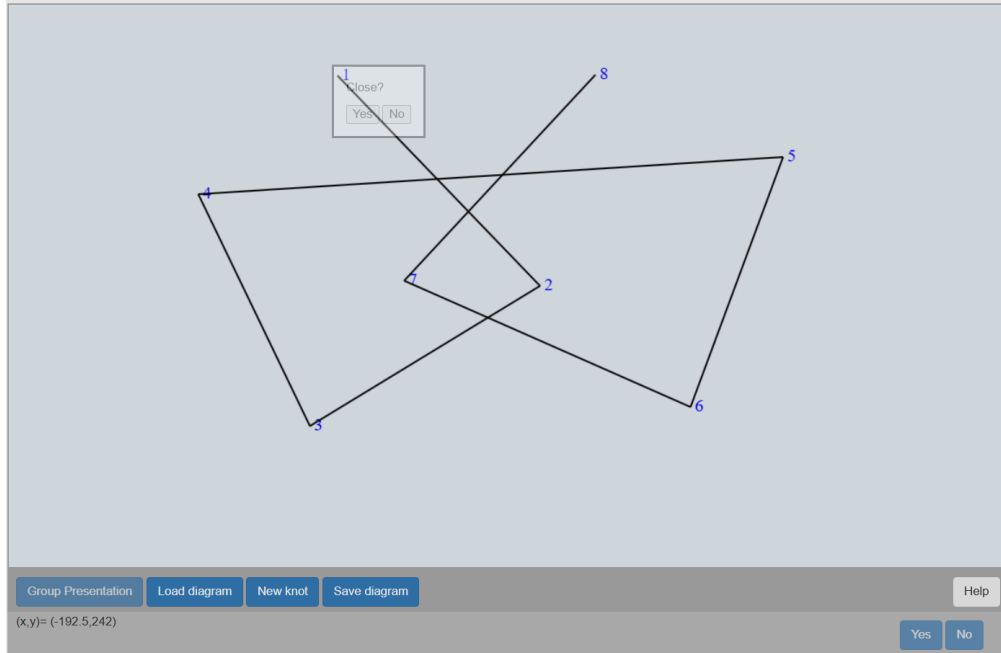


FIGURE 1. Knot diagram creation. The user is prompted with a “Close?” dialog when hovering over the first vertex to seamlessly connect the final segment and complete the closed loop.

1.3. Group Presentation Calculation.

- Once the crossing choices are resolved, the user can use the “Group Presentation” button, which is the computational core of the tool.
- When clicked, it processes the combinatorial data locally using the Python library SymPy. It algorithmically assembles the relators, applies Fox product rules, performs the abelianization, and computes the symbolic determinant.
- Instantly after executing the computation, the interface displays the full Wirtinger presentation, the Alexander polynomial, and the knot determinant directly below the drawing canvas.
- For knots with a high number of crossings, the resulting algebraic representation can become exceedingly large. To accommodate this, the output container features a scroll bar. This ensures that the user can smoothly navigate through the entire set of generators and relations without distorting the page layout.

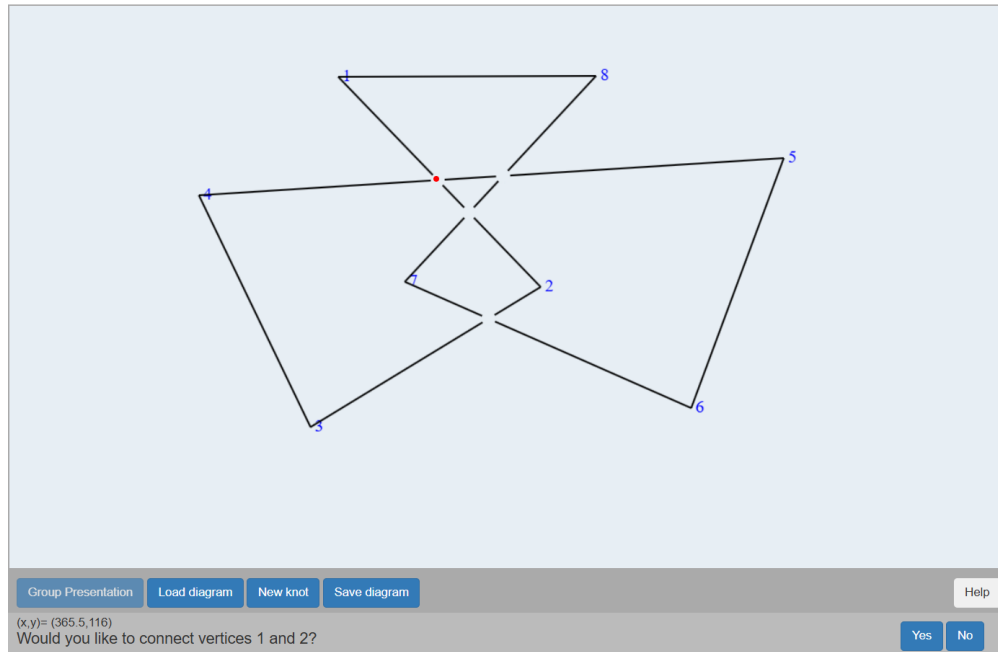


FIGURE 2. Intersection detection and resolution. The application highlights the active crossing with a red dot and prompts the user at the bottom of the screen to specify the over-passing segment.

- A “Save invariants” utility button is available, allowing the user to copy the generated mathematical output directly to their clipboard for immediate use in external documents.

1.4. Diagram Persistence.

- The “Save diagram” button allows users to preserve their work by exporting a text file containing both the vertex coordinates and the crossing choices.
- The “Load diagram” button imports these saved files, enabling users to render previously drawn knots and continue their analysis at any time.

1.5. Accessing the Application.

- Click the link <https://tacioli.github.io/Knot-Group-Presentation/> to use the live application.
- Go to the page <https://github.com/Tacioli/Knot-Group-Presentation> to access the complete source code repository.

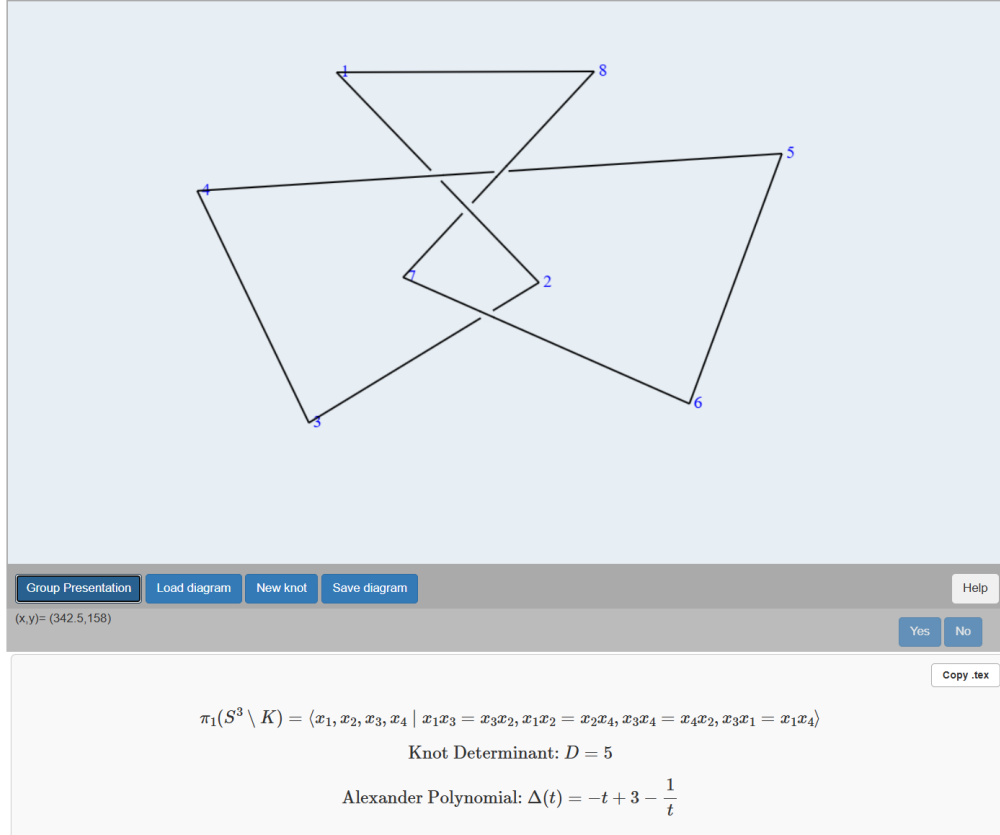


FIGURE 3. Group presentation calculation. Once all crossings are resolved, the application outputs the full Wirtinger presentation, the knot determinant, and the Alexander polynomial directly below the canvas.

2. USER-FRIENDLY INTERFACE

- The “New knot” button clears the canvas and resets all variables for a fresh drawing.
- The “Help” button toggles a visibility panel containing basic instructions for new users directly on the screen.

3. ADDITIONAL TECHNICAL DETAILS

- Developed using PyScript, the tool seamlessly integrates an interactive frontend with Python robust mathematical libraries.
- While an internet connection is required to load the web framework, all symbolic computations and matrix operations are executed entirely locally within the user web browser. This architecture eliminates the need for dedicated computational servers.

- When saving or loading diagram files, ensure they are in a `.txt` format and do not contain blank lines.